

Angular 22

Del setup al consumo de APIs REST

Del navegador vacio al frontend que consume la API de BiblioUTEQ

Docente: Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D.

Asignatura: Aplicaciones Web | Unidad IV --- Cliente enriquecido y REST

Stack: Angular 22.0.5 | Node.js 24 LTS | TypeScript 5.9 | IntelliJ IDEA Ultimate

Proyecto conductor: BiblioUTEQ --- Sistema de Gestion de Biblioteca Universitaria

Objetivos de aprendizaje

- Reconocer que es Angular y por que es el framework de referencia del curso para el cliente enriquecido.
- Instalar el entorno completo desde cero: Node.js LTS, Angular CLI e IntelliJ IDEA Ultimate.
- Crear un proyecto Angular con la CLI, comprender su estructura y ejecutarlo en el navegador.
- Generar componentes y servicios con la CLI aplicando los principios de responsabilidad unica.
- Consumir con HttpClient los endpoints REST del backend BiblioUTEQ (GET, POST, PUT, DELETE).
- Manejar errores, configurar interceptores para autenticacion, y separar la URL del backend por entorno.

Que es Angular

Angular es una plataforma de código abierto para construir aplicaciones web de una sola página (SPA), mantenida por Google y basada en TypeScript. Se distribuye bajo licencia MIT y cubre desde el andamiaje del proyecto hasta el enrutamiento, los formularios, el consumo de APIs y las pruebas.

Componentes

Piezas reutilizables que combinan HTML, CSS y lógica en TypeScript. Son la unidad básica de la interfaz.

Servicios e Inyección

Encapsulan reglas y acceso a datos. Se inyectan a los componentes por medio de la inyección de dependencias.

Enrutador y HttpClient

El enrutador cambia de vista sin recargar; HttpClient consume APIs REST devolviendo Observables de RxJS.

Angular no es lo mismo que AngularJS

AngularJS (versiones 1.x) llegó al fin de vida el 31 de diciembre de 2021 y no es compatible con el actual Angular. El actual Angular es una reescritura completa que comenzó en la versión 2 y se distribuye bajo el nombre corto Angular. Todo material que aparezca en la red bajo el nombre AngularJS o con código en \$scope es obsoleto y no debe seguirse en este curso.

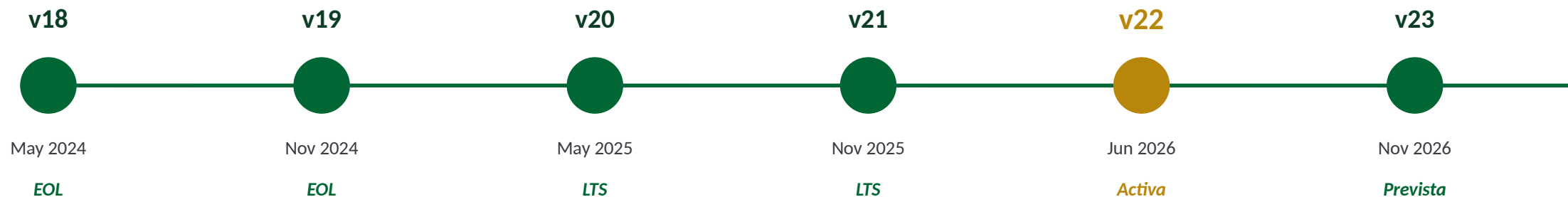
Angular frente a React y Vue

Aspecto	Angular 22	React 19	Vue 3
Tipo	Framework opinado (completo)	Librería de UI	Framework progresivo
Lenguaje base	TypeScript (obligatorio)	JavaScript o TypeScript	JavaScript o TypeScript
Enrutador oficial	Si (Router)	No (react-router-dom externo)	Si (Vue Router)
CLI oficial	Si (Angular CLI)	No (Vite, Next, CRA obsoleto)	Si (create-vue)
HTTP oficial	Si (HttpClient)	No (fetch o axios)	No (fetch o axios)
Reactividad	Signals (v22)	Hooks y estado	Reactivo por proxy
Curva inicial	Alta	Media	Media

La eleccion de Angular en este curso obedece a que trae el conjunto completo listo para trabajar (CLI, HttpClient, Router, formularios y pruebas) y a que su TypeScript obligatorio refuerza los habitos de tipado que se ensenan en la carrera.

Historia y versiones (a julio de 2026)

Angular sigue un cadence semestral de versiones mayores. Cada version esta soportada 18 meses: seis en soporte activo y doce en LTS. Para PPA 2026-2027 la version de referencia es Angular 22, publicada el 3 de junio de 2026.



Version del curso: Angular 22

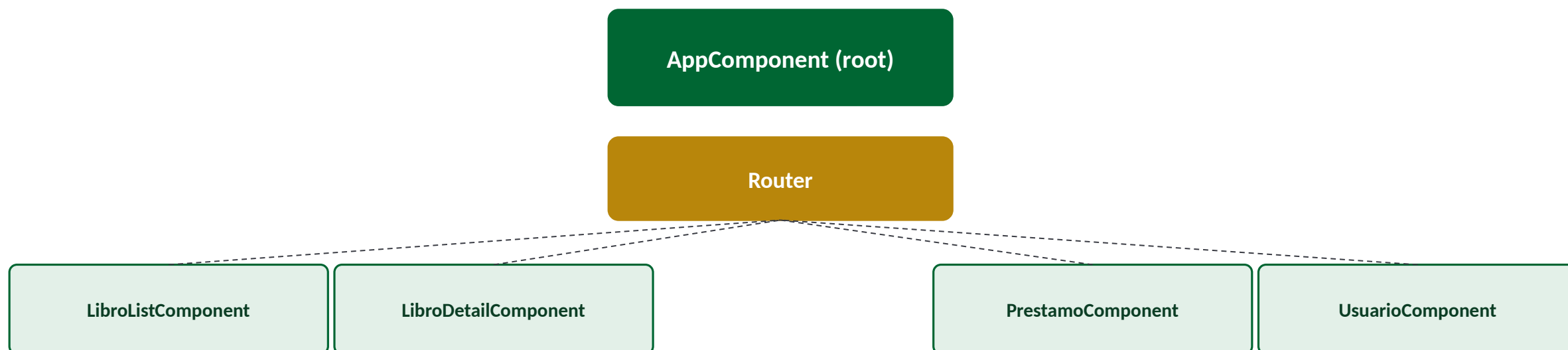
Publicada el 3-jun-2026. Trae Signal Forms estables, componentes selectorless y compilacion estricta con TypeScript 5.9. El patch actual es 22.0.5.

Comando para verificar la version

`ng version` imprime en consola la version de Angular CLI, el compilador @angular/compiler-cli, RxJS, TypeScript y el runtime de Node.js.

Arquitectura de una app Angular

Toda aplicación Angular se compone de una jerarquía de componentes con servicios y un enrutador que decide cuál componente renderiza cada URL. El siguiente esquema muestra la relación entre las piezas fundamentales.



Servicios inyectables: LibroService | PrestamoService | UsuarioService | AuthService

Cada uno usa HttpClient para hablar con /api/libros, /api/prestamos, /api/usuarios y /api/auth del backend BiblioUTEQ.

Requisitos del entorno

Para trabajar con Angular 22 se requiere una version reciente de Node.js LTS, el gestor de paquetes npm que viene con el, Angular CLI instalado globalmente y un IDE. La eleccion oficial del curso es IntelliJ IDEA Ultimate.

Node.js 24 LTS

Node 24 es la linea Active LTS a julio de 2026 (soporte hasta abr-2028). Trae npm 11 y V8 actualizado. Node 22 tambien es soportado pero ya en Maintenance LTS.

npm 11 o superior

npm es el gestor de paquetes de Node. Al instalar Node queda instalado. Permite ejecutar los scripts de package.json y bajar dependencias del registro publico.

Angular CLI 22

Se instala globalmente con `npm install -g @angular/cli`. Provee el comando `ng` para crear proyectos, generar componentes y correr el servidor de desarrollo.

IntelliJ IDEA Ultimate

Version 2024.x o superior. La edicion Community NO trae soporte para Angular; se debe usar Ultimate, gratuita para estudiantes con licencia academica.

Instalar Node.js 24 LTS

Windows 10/11

- Descargar el instalador .msi para 64 bits desde el sitio oficial nodejs.org, seccion Downloads > LTS.
- Ejecutar el .msi con doble clic. Aceptar el acuerdo de licencia y dejar la ruta por defecto.
- Marcar la casilla Add to PATH y la de instalar las Chocolatey Tools (opcional pero recomendado).
- Reiniciar la terminal (o cerrar y volver a abrir PowerShell) para que el PATH se recargue.

Ubuntu 24.04 LTS

- Usar NodeSource (paquete oficial). No usar el paquete apt de Ubuntu que trae versiones muy antiguas.
- Ejecutar curl del script de setup para Node 24 con sudo bash.
- Instalar el paquete nodejs con sudo apt-get install -y nodejs (npm viene incluido).
- Verificar la instalacion con node --version y npm --version.

terminal

```
# Windows (PowerShell): descargar de nodejs.org y ejecutar el .msi
# Ubuntu / Debian: script NodeSource para Node 24 LTS
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -

sudo apt-get install -y nodejs
```


Verificar Node y npm

Una vez instalado, el aprendiz confirma con dos comandos que Node y npm quedaron en el PATH y que las versiones son compatibles con Angular 22. Salidas esperadas:

terminal

```
$ node --version
```

```
v24.11.0
```

```
$ npm --version
```

```
11.5.2
```

Versiones minimas requeridas por Angular 22

Angular 22 soporta oficialmente Node.js ^20.19.0 || ^22.12.0 || ^24.0.0 y npm ^11.0.0. Si el aprendiz instalo por error una version Current impar (25 o 26 en 2026), tambien funciona pero no es LTS y no se recomienda para produccion. Cualquier version inferior a Node 20 hara fallar ng new con un mensaje del tipo 'Node.js version ... is not supported'.

Instalar Angular CLI

Angular CLI se instala globalmente con npm. Con -g queda accesible como el comando ng en cualquier directorio de la maquina. Para PPA 2026-2027 se fija la version 22, que se corresponde con la version de Angular del curso.

terminal

```
# Instalacion global de Angular CLI 22  
npm install -g @angular/cli@22
```

```
# Verificacion  
ng version
```

La salida esperada de ng version es una tabla ASCII con el logo de Angular y las versiones de CLI, compilador, RxJS, TypeScript y Node. Si ng version devuelve un error de politica de ejecucion en PowerShell, el aprendiz debe habilitar los scripts con:

powershell

```
# PowerShell: permitir ejecucion de scripts firmados para el usuario actual  
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Instalar IntelliJ IDEA Ultimate

- Descargar el instalador de la pagina jetbrains.com/idea (opcion Ultimate).
- Iniciar sesion con la cuenta de estudiante (correo institucional @uteq.edu.ec) para activar la licencia academica gratuita.
- Al primer arranque, asegurarse de que los plugins Angular and AngularJS, JavaScript and TypeScript, y Node.js esten habilitados (File > Settings > Plugins).
- Configurar el SDK de Node.js (File > Settings > Languages and Frameworks > Node.js) apuntando al binario recién instalado.
- Configurar el JDK de Java 21 para el backend Spring Boot (File > Project Structure > SDKs).

Crear el proyecto biblioteq-front

Con Node y Angular CLI instalados, el aprendiz crea el proyecto frontend del monorepo BiblioUTEQ con un solo comando. La CLI hace todo el andamiaje: genera archivos, instala dependencias y deja el proyecto listo para ejecutarse.

terminal

```
# Situarse en la raiz del monorepo  
cd biblioteq/
```

```
# Crear el proyecto frontend con la CLI  
ng new frontend --style=css --routing=true --ssr=false --skip-git=true
```

```
# Ingresar al proyecto e instalar dependencias
```

Explicacion de los banderines de ng new

--style=css usa CSS puro (no SCSS) coherente con la Unidad I. --routing=true crea el Router configurado. --ssr=false omite Server-Side Rendering (se cubre en Unidad IV avanzada). --skip-git=true evita crear un repositorio Git dentro del monorepo que ya tiene su propio Git en la raiz.

Estructura del proyecto

La CLI genera una estructura estandar que sigue las convenciones del equipo de Angular. Cada carpeta y cada archivo tiene un rol claro.

```
tree
```

```
frontend/  
|-- src/  
|   |-- app/  
|       |-- app.component.ts  
|       |-- app.component.html  
|       |-- app.component.css  
|       |-- app.config.ts  
|       |-- app.routes.ts  
|   |-- assets/  
|   |-- index.html  
|   |-- main.ts  
|   |-- styles.css
```

src/app/ --- Código TypeScript de la aplicación.

app.config.ts --- Configura providers globales (HttpClient, Router).

app.routes.ts --- Rutas del enrutador: URL a Componente.

styles.css --- Estilos globales del proyecto. Aquí se importa el estilos.css compartido de BiblioUTEQ.

angular.json --- Configuración del build (assets, budgets).

package.json --- Dependencias y scripts npm.

Ejecutar la aplicacion

El comando `ng serve` compila el proyecto, inicia un servidor de desarrollo en `localhost:4200` y activa la recarga en caliente: cualquier cambio en el código actualiza el navegador sin perder el estado.

terminal

```
# Desde la carpeta frontend/  
ng serve --open  
  
# Salida esperada (abreviada):  
# Application bundle generation complete. [3.421 seconds]  
# Watch mode enabled. Watching for file changes...  
# Local: http://localhost:4200/
```

Que hace la CLI internamente

Compila TypeScript a JavaScript (esbuild), agrupa módulos, procesa CSS, sirve por HTTP con recarga en caliente y expone sourcemaps para depurar en el navegador.

Puerto en uso

Si el 4200 está ocupado, se pasa `--port 4300` al comando. El equipo BiblioUTEQ usa 4200 para el frontend y 8080 para la API.

Abrir el proyecto en IntelliJ IDEA

- File > Open > seleccionar la carpeta frontend/ recién creada por ng new.
- Confirmar la opción Trust Project cuando IntelliJ lo solicite.
- IntelliJ detecta automáticamente el proyecto Angular gracias al angular.json en la raíz.
- En View > Tool Windows > Angular se abre el panel con la lista de componentes, servicios y módulos del proyecto, útil para navegar.
- Crear una configuración de Run > Edit Configurations > + > npm, con script 'start' del package.json, para arrancar ng serve con un solo botón verde.
- Habilitar el ESLint automático en Settings > Languages and Frameworks > TypeScript > TSLint/ESLint para que resalte errores en tiempo real.

Componentes: la unidad basica

Un componente en Angular es una clase TypeScript decorada con `@Component` que asocia una plantilla HTML, unos estilos CSS y una logica. Es la unidad reutilizable de la interfaz: un boton, una tabla, una pagina completa.

typescript

```
// biblioteca-front/src/app/libro-list.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-libro-list',
  standalone: true,
  template: `
    <h2>Catalogo de libros</h2>

    <p>Total en catalogo: {{ total }}</p>
  `
})
```

El decorador `@Component` enlaza la clase con su HTML, su CSS y un selector `<app-libro-list>`. Cualquier plantilla que incluya esa etiqueta renderizara este componente. La bandera `standalone:true`, que es el default en Angular 22, permite usar el componente sin declararlo en un `NgModule`.

```
styles: [ 'h2 { color: #006633; }' ]
```


Generar un componente con la CLI

En vez de escribir el componente a mano, la CLI lo genera con el andamiaje correcto: archivo .ts, .html, .css, .spec.ts, y lo registra automaticamente en las importaciones si es hijo de la app.

terminal

```
# Sintaxis larga  
ng generate component libro-list
```

```
# Forma abreviada (equivalente)  
ng g c libro-list
```

```
# Con opciones
```

Archivos creados

libro-list.component.ts (clase con @Component), .html (template), .css (estilos scoped) y .spec.ts (esqueleto de pruebas con Jasmine).

Buena practica

Agrupar componentes por característica en subcarpetas: libros/, prestamos/, usuarios/. La opcion libros/libro-list crea el componente ya en esa carpeta.

Data binding

Data binding es el mecanismo por el que Angular sincroniza el modelo (la clase TypeScript) con la vista (el HTML). Hay cuatro variantes esenciales.

Interpolacion

```
{{ libro.titulo }}
```

Muestra el valor de una expresion en el DOM.

Property binding

```
[value]="libro.titulo"
```

Asigna un valor a una propiedad del elemento HTML.

Event binding

```
(click)="onGuardar()"
```

Enlaza un evento del DOM a un metodo de la clase.

Two-way binding

```
[(ngModel)]="libro.titulo"
```

Sincronia bidireccional; requiere FormsModule.

Control flow: @if y @for

Desde Angular 17 se recomienda el nuevo control flow con las etiquetas @if, @for y @switch, mas rapido y mas legible que las directivas historicas *ngIf y *ngFor.

template.html

```
<!-- Renderizado condicional -->
@if (libro) {

  <h2>{{ libro.titulo }}</h2>

} @else {

  <p>No hay libro seleccionado.</p>

}

<!-- Iteracion sobre una lista -->
@for (l of libros; track l.id) {
```

@for requiere track

El identificador track (por ejemplo track l.id) es obligatorio: sin el Angular no puede optimizar los cambios en la lista y arroja error de compilacion.

```
} @empty {
```

Servicios e inyeccion de dependencias

Un servicio es una clase TypeScript decorada con `@Injectable` que encapsula logica que no pertenece a la vista: reglas de negocio, acceso a la API, cache, autenticacion. Se inyecta a los componentes en el constructor o con la funcion `inject()`.

typescript

```
// bibliotek-front/src/app/libro.service.ts
import { Injectable, inject } from '@angular/core';

import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })

export class LibroService {

  private http = inject(HttpClient);

  private readonly api = 'http://localhost:8080/api/libros';
```

```
  listar() {
```

La bandera `providedIn: 'root'` registra el servicio como singleton en el inyector raiz: existe una sola instancia para toda la aplicacion. La funcion `inject()` de Angular 14+ reemplaza al constructor con parametros y hace mas legible el servicio.

HttpClient: configurar el proveedor

HttpClient es el modulo oficial de Angular para hablar con APIs por HTTP. Devuelve Observables de RxJS, lo que permite componer, cancelar y transformar peticiones. Antes de usarlo hay que registrarlo en la configuracion global.

typescript

```
// bibliotek-front/src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';

import { provideRouter } from '@angular/router';

import { provideHttpClient, withInterceptors } from '@angular/common/http';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([]))
  ]
}
```

Con `provideHttpClient()` cualquier servicio de la app puede inyectar `HttpClient` y hacer peticiones. La opcion `withInterceptors([])` queda preparada para anadir interceptores (por ejemplo, uno que agregue el token JWT); se cubre mas adelante.

Modelar la respuesta de la API

Antes de consumir, se define una interface TypeScript que refleje el JSON que responde el backend. Esto brinda autocompletado en IntelliJ y detecta en compilación inconsistencias entre frontend y backend.

libro.model.ts

```
// bibliotek-front/src/app/libro.model.ts
export interface Libro {

  id: number;

  isbn: string;

  titulo: string;

  autor: string;

  editorial: string;

  anio: number;

  categoriaId: number;

  disponibles: number;

}
```

respuesta JSON

```
// GET /api/libros/1 --- respuesta
{

  "id": 1,

  "isbn": "978-84-376-0494-7",

  "titulo": "Cien anos de soledad",

  "autor": "Gabriel Garcia Marquez",

  "editorial": "Sudamericana",

  "anio": 1967,

  "categoriaId": 3,

  "disponibles": 4

}
```

Consumir la API: GET listar

El servicio LibroService encapsula la petición GET al endpoint /api/libros. Devuelve un Observable<Libro[]> que el componente consumirá.

libro.service.ts

```
// bibliotec-front/src/app/libro.service.ts
import { Injectable, inject } from '@angular/core';

import { HttpClient } from '@angular/common/http';

import { Observable } from 'rxjs';

import { Libro } from '../libro.model';

import { environment } from '../environments/environment';

@Injectable({ providedIn: 'root' })
export class LibroService {
  private http = inject(HttpClient);

  private readonly api = `${environment.apiUrl}/libros`;
}
```

Componente que consume el servicio

libro-list.component.ts

```
// bibliotecq-front/src/app/libro-list.component.ts
import { Component, inject, signal, OnInit } from '@angular/core';

import { LibroService } from '../libro.service';
import { Libro } from '../libro.model';

@Component({
  selector: 'app-libro-list',
  standalone: true,
  template: `
    <h2>Catalogo</h2>

    @for (l of libros(); track l.id) {
      <p>{{ l.titulo }} --- {{ l.autor }}</p>
    } @empty { <p>Cargando...</p> }
  `
})
```


Alternativa con async pipe

El async pipe se suscribe al Observable en la propia plantilla y se desuscribe al destruir el componente. Evita el código boilerplate del subscribe y previene fugas de memoria.

con async pipe

```
// bibliotecq-front/src/app/libro-list.component.ts
import { Component, inject } from '@angular/core';

import { AsyncPipe } from '@angular/common';

import { LibroService } from '../libro.service';
```

```
@Component({
  selector: 'app-libro-list',
  standalone: true,
  imports: [AsyncPipe],
  template: `
```

```
    @for (l of libros$ | async; track l.id) {
```

Crear un recurso: POST

POST /api/libros

```
// libro.service.ts (extracto)

crear(libro: Libro): Observable<Libro> {
  return this.http.post<Libro>(this.api, libro);
}

// libro-form.component.ts
import { Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  standalone: true,
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="nuevo.titulo" name="titulo" placeholder="Titulo"/>`
})
```

Actualizar y eliminar: PUT y DELETE

CRUD completo

// libro.service.ts (extracto)

```
actualizar(id: number, libro: Libro): Observable<Libro> {  
  return this.http.put<Libro>(`${this.api}/${id}`, libro);  
}
```

```
eliminar(id: number): Observable<void> {  
  return this.http.delete<void>(`${this.api}/${id}`);  
}
```

```
buscarPorId(id: number): Observable<Libro> {  
  return this.http.get<Libro>(`${this.api}/${id}`);  
}
```

Manejo de errores con catchError

HttpClient emite el error del Observable cuando el backend responde con código 4xx o 5xx, o cuando hay un fallo de red. El operador catchError de RxJS captura el error y permite reaccionar sin romper el flujo.

catchError

```
// libro.service.ts (extracto)
import { catchError, throwError } from 'rxjs';

import { HttpResponseResponse } from '@angular/common/http';

listar(): Observable<Libro[]> {

  return this.http.get<Libro[]>(this.api).pipe(

    catchError((err: HttpResponseResponse) => {

      console.error('Error al listar libros', err.status, err.message);

      // se puede transformar en un mensaje amigable
      const msg = err.status === 0

        ? 'No se pudo contactar al servidor.'
```

Interceptor HTTP para JWT

Un interceptor es una funcion que ejecuta Angular antes de enviar cada peticion HTTP. El uso mas comun es agregar el token JWT de autenticacion a la cabecera Authorization sin repetir codigo en cada servicio.

auth.interceptor.ts

```
// bibliotec-front/src/app/auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

import { inject } from '@angular/core';

import { AuthService } from '../auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {

  const token = inject(AuthService).getToken();

  if (!token) return next(req);

  const withAuth = req.clone({
    headers: {
      Authorization: `Bearer ${token}`
    }
  });

  return next(withAuth);
};
```

El interceptor se registra en app.config.ts: provideHttpClient(withInterceptors([authInterceptor])). A partir de ese momento, todas las peticiones salientes llevan el token si el usuario esta autenticado.

URLs por entorno: environments

En vez de cablear la URL del backend en cada servicio, se centraliza en archivos environment por entorno. Angular sustituye automáticamente al compilar.

desarrollo

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api'
};
```

produccion

```
// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://biblioteq.uteq.edu.ec/api'
};
```

Habilitar file replacement en angular.json

La sustitucion no es automatica en Angular 22: en angular.json, en configurations > production > fileReplacements, se declara que src/environments/environment.ts se reemplaza por environment.prod.ts al construir con ng build --configuration=production.

CORS: origen cruzado

El navegador bloquea las peticiones de un origen a otro por seguridad. Como el frontend Angular corre en localhost:4200 y el backend Spring Boot en localhost:8080, ambos son orígenes distintos y el navegador rechaza la petición salvo que el backend lo autorice vía CORS.

Sintoma

En la consola del navegador aparece: 'Access to XMLHttpRequest at ... has been blocked by CORS policy: No Access-Control-Allow-Origin header is present'.

Solucion

Se resuelve en el servidor, no en Angular. El backend Spring Boot debe agregar cabeceras Access-Control-Allow-Origin y responder a preflight (OPTIONS).

backend Spring

```
// backend Spring Boot: config/SecurityConfig.java
@Bean CorsConfigurationSource corsConfigurationSource() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("http://localhost:4200"));
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
}
```

Enrutamiento basico

El Router de Angular asocia URLs a componentes. La configuracion vive en app.routes.ts y el HTML principal declara donde se renderiza el componente activo con <router-outlet>.

app.routes.ts

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';

import { LibroListComponent }
  from './libros/libro-list.component';

import { LibroDetailComponent }
  from './libros/libro-detail.component';

export const routes: Routes = [
  { path: '', redirectTo: 'libros',
    pathMatch: 'full' },
  { path: 'libros', component: LibroListComponent },
  { path: 'libros/:id', component: LibroDetailComponent }
```

app.component.html

```
<!-- src/app/app.component.html -->
<header>

  <a routerLink="/libros">Catalogo</a>

  <a routerLink="/prestamos">Prestamos</a>

</header>

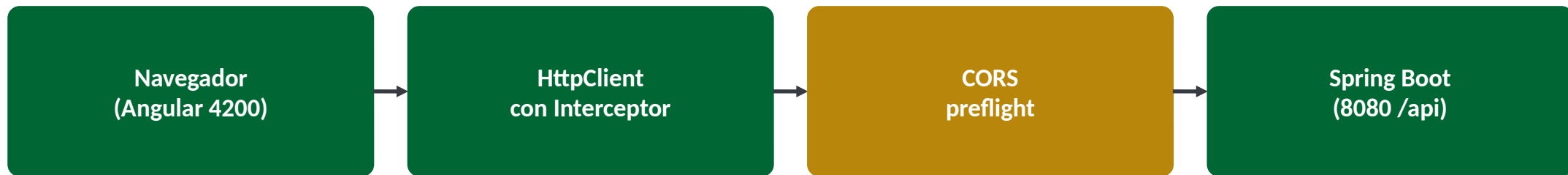
<main>

  <router-outlet></router-outlet>

</main>
```


Integracion completa en BiblioUTEQ

Con las piezas anteriores ya integradas, el flujo del catalogo publico de BiblioUTEQ queda funcional. El aprendiz puede navegar a /libros, ver la lista y hacer clic para ver el detalle.



arranque completo

```
# 1) Levantar el backend (otra terminal, otra carpeta)
cd biblioteq/backend-java && mvn spring-boot:run
```

```
# 2) Levantar el frontend
cd biblioteq/frontend && ng serve --open
```

Build para produccion

Cuando la aplicacion esta lista para desplegarse, ng build produce un paquete optimizado (minificado, con tree-shaking y AOT) que se sirve como archivos estaticos desde nginx, Apache o un CDN.

terminal

```
# Build de produccion (usa environment.prod.ts)  
ng build --configuration=production
```

```
# Salida en dist/frontend/browser/  
# Subir el contenido de dist/frontend/browser al servidor estatico
```

Optimizaciones aplicadas

Compilacion AOT (ahead-of-time), tree-shaking para eliminar codigo no usado, minificacion y compresion de assets, hashing de nombres para invalidacion de cache.

Budgets del build

Angular.json define limites de tamano (bundle inicial 500 kB, componente 4 kB). El build falla si se superan; obliga a mantener la aplicacion ligera.

Referencias

- [1] Google LLC. Angular Documentation, v22.0.5. <https://angular.dev> (jul. 2026).
- [2] OpenJS Foundation. Node.js Documentation, v24 LTS. <https://nodejs.org/docs/latest-v24.x> (jul. 2026).
- [3] Microsoft. TypeScript Handbook 5.9. <https://www.typescriptlang.org/docs/> (2026).
- [4] ReactiveX. RxJS 7.8 Documentation. <https://rxjs.dev> (2024).
- [5] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. Tesis, UC Irvine, 2000.
- [6] Fielding, R.; Nottingham, M.; Reschke, J. HTTP Semantics. IETF RFC 9110. Jun. 2022. doi:10.17487/RFC9110.
- [7] Nottingham, M.; Wilde, E. Problem Details for HTTP APIs. IETF RFC 7807. Mar. 2016. doi:10.17487/RFC7807.
- [8] Jones, M.; Bradley, J.; Sakimura, N. JSON Web Token (JWT). IETF RFC 7519. May 2015. doi:10.17487/RFC7519.
- [9] OWASP Foundation. OWASP Top 10:2021. <https://owasp.org/Top10/>
- [10] JetBrains. IntelliJ IDEA Ultimate --- WebStorm Documentation for Angular. <https://www.jetbrains.com/help/idea/angular.html> (2026).
- [11] W3C. Cross-Origin Resource Sharing (CORS). Living standard, WHATWG Fetch. <https://fetch.spec.whatwg.org/>
- [12] Freeman, A. Pro Angular: Build Powerful and Dynamic Web Apps. 5.a ed. Apress, 2023. isbn 978-1-4842-9426-1.

Angular 22 --- del setup al consumo REST

Que se logro

1. Entorno completo instalado: Node.js 24 LTS + Angular CLI 22 + IntelliJ IDEA Ultimate.
2. Proyecto frontend/ creado y corriendo en localhost:4200 con recarga en caliente.
3. Componentes y servicios generados con la CLI, con inyeccion de dependencias.
4. CRUD completo consumiendo /api/libros con HttpClient, tipado por interfaces.
5. Interceptor JWT, manejo de errores, environments por entorno y CORS resuelto.

Que sigue

Formularios reactivos, guards de rutas para proteger vistas por rol, pruebas unitarias con Vitest, y despliegue detras de nginx. Todo se cubre en los capítulos siguientes de la Unidad IV.